Laborator 3 - Mijloace de organizare a proiectului. Controlul versiunii

Cum să ne organizăm pentru a atinge obiectivele definite în SRS conform planurilor din SDD?

În managementul de proiect, există o serie de utilitare de organizare a lucrului în proiecte, precum:

- **WBS**
- **Grafic Gantt**

WBS (Work Breakdown Structure)

- structură arborescentă cu rolul de a determina și de a grupa **sarcinile proiectului** în funcție de categoria din care fac parte
- ajută la organizarea proiectului prin clarificarea obiectivelor, a **elementelor de lucru** mai generale (*work packages*) și mai specifice (*work elements*)
- pe baza WBS, se pot face estimări de durată, de resurse umane și de buget → *suport pentru construirea diagramei Gantt*
- ajută la identificarea relațiilor de precedență dintre elementele de lucru (care elemente de lucru sunt independente, care sunt restricționate să aibă loc înainte de altele, etc.) - elemente vizibile în diagrama Gantt.
- WBS reprezinta graful acțiunilor necesare unui proiect grupând acțiunile în categorii de interes
- oferă suport pentru identificarea tuturor acțiunilor necesare și prioritizarea lor în funcție de efort, cost, durată etc.

Modul de construire a WBS

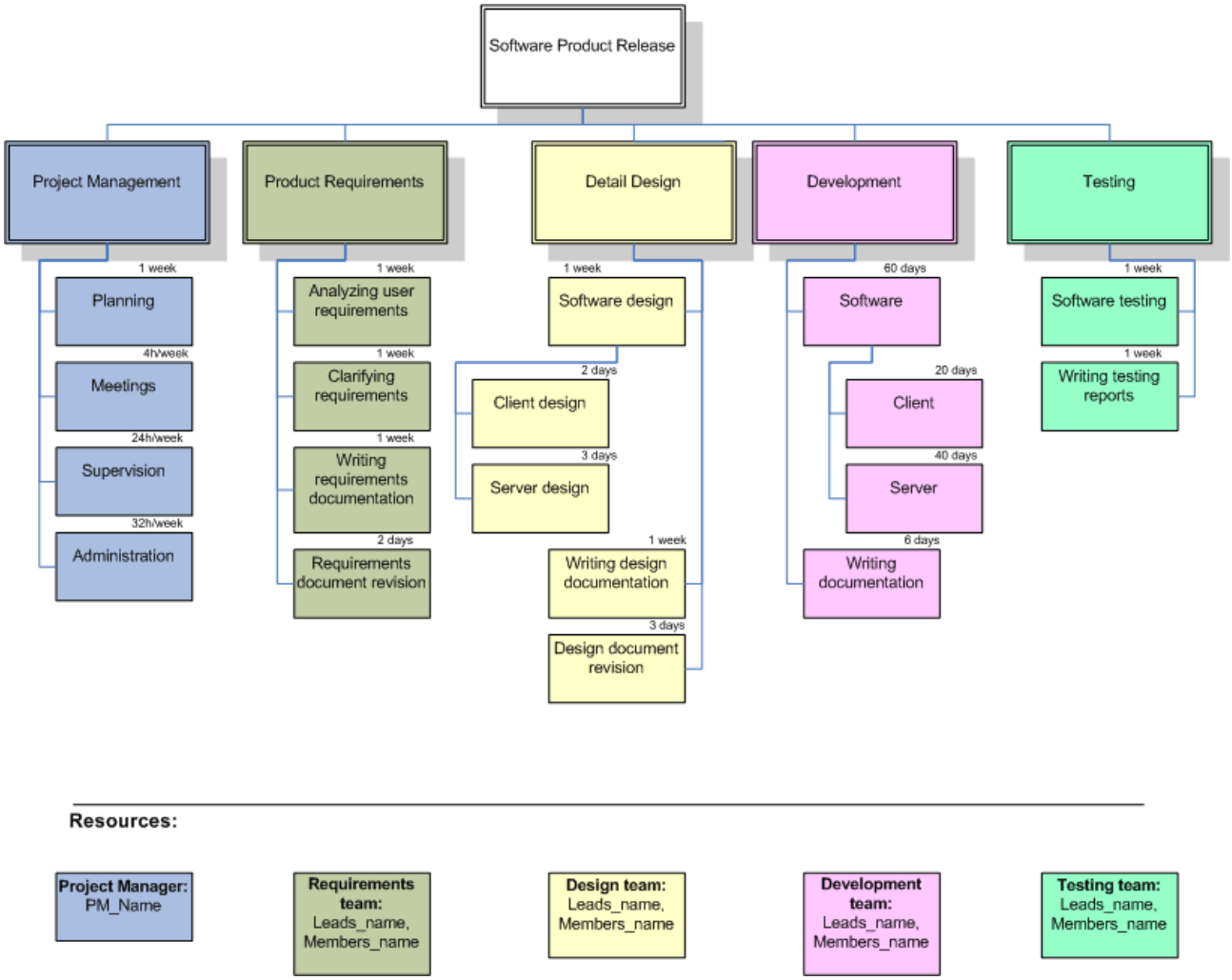
- nodurile arborelui conțin elementele de lucru, care pot fi produse, date, servicii
- rădăcina arborelui coincide cu obiectivul proiectului: sistemul software de dezvoltat
- nivelul al doilea al arborelui: subsistemele componente ale sistemului
- tot așa în adâncimea arborelui, în sensul că orice nod părinte înseamnă un element de lucru general, iar copiii reprezintă elemente de lucru componente ale lui
- deci, se detaliază efortul de dezvoltare a proiectului, de la obiectivul general la elemente de lucru și sarcini practice.
- nivelul de detaliere al unui WBS nu este foarte ridicat; astfel, adâncimea arborelui nu depășește, adesea, mai mult de *4-5 niveluri*.

Reguli

- un WBS trebuie să cuprindă prin elementele sale **tot** efortul necesar dezvoltării proiectului (100% din muncă)
- elementele de muncă de pe același nivel din arbore nu se acoperă total sau parțial (nu sunt *overlapping*)
- frunzele din arbore au adesea asociate resurse, durate și costuri.

Tool-uri de creare a WBS-urilor: WBS Schedule Pro [http://www.criticaltools.com/], Online Visual Paradigm [https://online.visual-paradigm.com/diagrams/features/work-breakdown-structure-software/], Planhammer.io [https://planhammer.io/wbs-chart-software.html]

Exemplu de diagramă WBS a unui proiect software:



Grafic Gantt

- grafic pentru **planificarea temporală** a sarcinilor proiectului
- construit pe baza activităților identificate în **WBS**
- indică data de început și de sfârșit a activităților din WBS și dependențele dintre activități:
  - start-to-start
  - finish-to-start
  - finish-to-finish
- întotdeauna se va ține cont de potențialele întârzieri și vor incluse în planificare sub o formă sau alta (review, delay, etc.)
- graficul Gantt este unul orientativ și starea sa se poate altera în timpul proiectului în funcție de situație
- acțiunile în grafic sunt trecute ca începand cel mai curând posibil, urmând să se asigure o marjă de eroare până la momentul cel mai târziu posibil
- este un tool bun pentru monitorizarea stării proiectului în timp, în funcție de gradul de realizare a acțiunilor până la un moment dat
- adesea, conține și următoarele elemente:
  - o linie verticală ce marchează timpul prezent
  - alte linii verticale cu milestone-urile proiectului
  - starea activităților exprimată prin procente (100% înseamnă că activitatea a fost realizată complet).

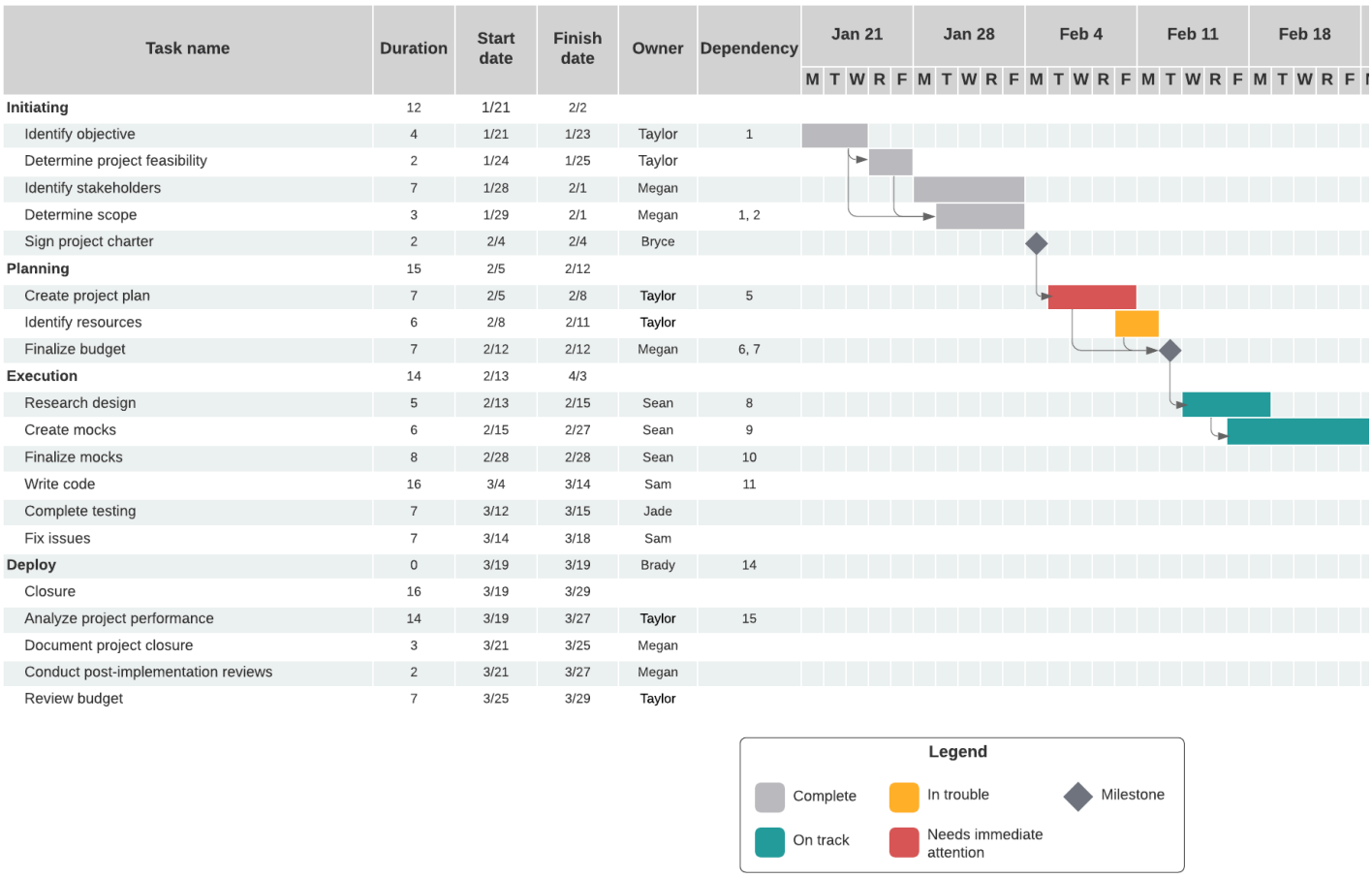
Organizarea unui proiect înseamnă parcurgerea etapelor:

1. Determinarea activităților proiectului (*WBS*)
2. Estimarea duratelor activităților și a consumului de resurse (astfel încât costul total să fie mai mic decât bugetul alocat)
3. Planificarea activităților în funcție de dependențe și de eventualele restricții legate de resurse (*Gantt*)
4. Stabilirea de *milestone-uri*.

Tool-uri de creare a diagramelor Gantt: Gantt Project [https://www.ganttproject.biz/], Team Gantt [https://www.teamgantt.com/free-gantt-chart-excel-template], Gantter [https://www.gantter.com/], Gantt Excel [https://www.ganttexcel.com/], Microsoft Project.

Exemplu de diagramă Gantt a unui proiect ce conține și milestones:

Task name	Duration	Start date	Finish date	Owner	Dependency
Initiating	12	1/21	2/2		
Identify objective	4	1/21	1/23	Taylor	1
Determine project feasibility	2	1/24	1/25	Taylor	
Identify stakeholders	7	1/28	2/1	Megan	
Determine scope	3	1/29	2/1	Megan	1, 2
Sign project charter	2	2/4	2/4	Bryce	
Planning	15	2/5	2/12		
Create project plan	7	2/5	2/8	Taylor	5
Identify resources	6	2/8	2/11	Taylor	
Finalize budget	7	2/12	2/12	Megan	6, 7
Execution	14	2/13	4/3		
Research design	5	2/13	2/15	Sean	8
Create mocks	6	2/15	2/27	Sean	9
Finalize mocks	8	2/28	2/28	Sean	10
Write code	16	3/4	3/14	Sam	11
Complete testing	7	3/12	3/15	Jade	
Fix issues	7	3/14	3/18	Sam	
Deploy	0	3/19	3/19	Brady	14
Closure	16	3/19	3/29		
Analyze project performance	14	3/19	3/27	Taylor	15
Document project closure	3	3/21	3/25	Megan	
Conduct post-implementation reviews	2	3/21	3/27	Megan	
Review budget	7	3/25	3/29	Taylor	



Milestone

- deadline de finalizare a unei etape importante și decisive a proiectului
- adesea presupune predarea unor livrabile către beneficiar
- în vederea respectării milestone-urilor fixate, adesea se impune luarea unor decizii cu impact major asupra evoluției proiectului.
- deși reprezintă puncte cheie în cadrul unui timeline, din varii motive acestea pot fi decalate dacă acest lucru ajută și nu au un impact negativ asupra proiectului.

Controlul versiunii (Git)

Sisteme pentru controlul versiunii [http://en.wikipedia.org/wiki/Revision_control] (*Version Control Systems* – **VCS** – sau *Source Code Management* – **SCM**) sunt aplicații care permit lucrul colaborativ pe diverse fișiere, în special fișiere cod sursă. Sistemele pentru controlul versiunii sunt practic obligatorii în cadrul unui proiect cu dezvoltatori multipli. Astfel de sisteme reține istoricul modificărilor efectuate de fiecare dezvoltator și folosesc comenzi specializate care să faciliteze transmiterea acestor modificări între dezvoltatori.

Exceptând sistemele de gestiune a surselor, prezentate mai detaliat în continuare, și alte aplicații folosesc versiuni:

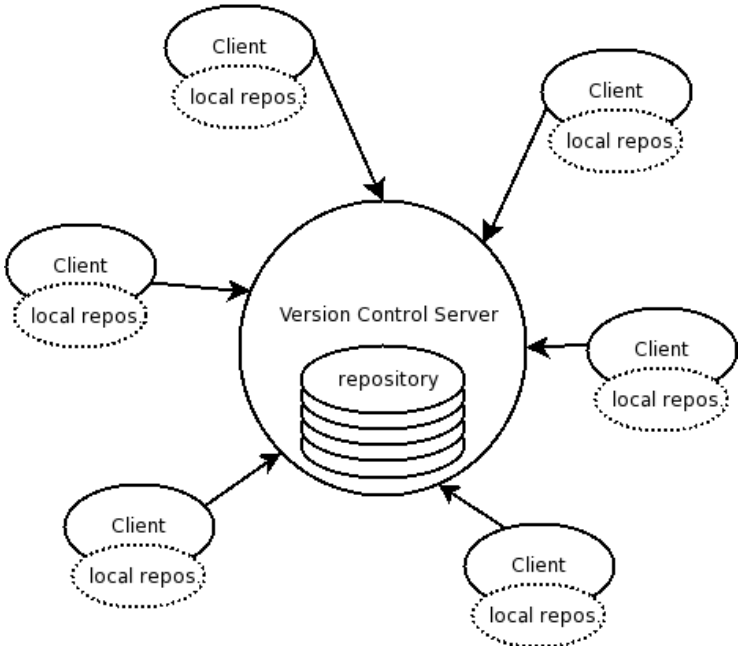
- Wiki Engines - pentru fiecare modificare se reține doar schimbările de la versiunea anterioară; orice modificare poate fi anulată;
- Google Docs - versiuni pentru fiecare modificare;
- MS Office, OpenOffice - permit atașarea unor numere de identificare pe documentele editate.

Principiul de funcționare a sistemelor de gestiune a codului este comun:

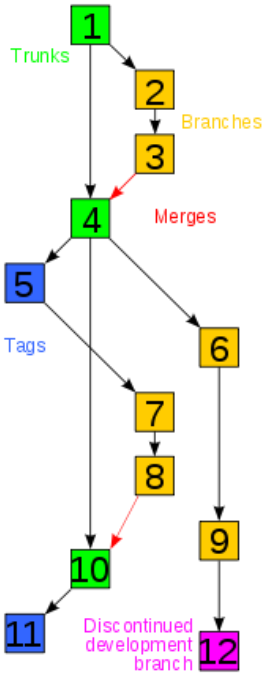
- Sursele sunt păstrate, de obicei, într-un **repository** (**depozit**) aflat pe un server accesibil tuturor dezvoltatorilor. Acest repository constituie mecanismul principal de sincronizare a surselor dezvoltatorilor.
- Fiecare dezvoltator obține o copie a repository-ului, denumită **copie locală**. Copia rezidă pe sistemul dezvoltatorului. Dezvoltarea se va realiza în acest director.
- Există două operații care permit comunicația cu repository-ul:
  - **update** sau **pull** înseamnă actualizarea copiei locale cu informațiile din repository; este posibil ca alți dezvoltatori să fi comis schimbări în repository;
  - **commit** sau **push** înseamnă transmiterea/comiterea modificărilor locale în repository; ceilalți dezvoltatori vor folosi comanda **update** pentru actualizarea copiei locale cu acele informații.

Câteva cuvinte cheie utile în lucrul cu sisteme de control al versiunii sunt (denumirile sunt în engleză pentru că aceasta este forma uzuală de utilizare):

- **repository** locul (centralizat) în care se găsesc sursele și informațiile de modificare;



- **working copy** copia locală a repository-ului folosită de un dezvoltator;
- **check-out** operația de creare a unei copii locale pornind de la repository (clonarea repository-ului);
- **commit** (vezi mai sus)
- **update** (vezi mai sus)
- **branch** o ramură de dezvoltare (un fork) care poate fi dezvoltată în paralel față de ramura principală de dezvoltare (vezi imaginea de mai jos);
- **trunk** ramura principală de dezvoltare (vezi imaginea de mai jos);
- **tag** o referință la un snapshot al surselor la un moment dat în timp;
- **merge** o operație de unificare a două seturi de schimbări petrecute distinct;
- **conflict** situație în care două modificări sunt efectuate din surse diferite și sistemul nu poate găsi o soluție de unificare (merge) a acestor schimbări; dezvoltatorul va trebui să rezolve conflictul fie prin unificarea schimbărilor, fie prin selectarea unei singure variante



Există două tipuri de sisteme pentru gestiunea codului sursă:

- sisteme centralizate (Subversion [http://subversion.tigris.org/], CVS [http://www.nongnu.org/cvs/], perforce [http://www.perforce.com/]);
- sisteme distribuite (Git [http://git-scm.com/], Darcs [http://darcs.net/], Mercurial [http://mercurial.selenic.com/wiki/], Bazaar [http://bazaar-vcs.org/]).

Detalii despre diferențele dintre acestea (mai degrabă între doi dintre cei mai cunoscuți reprezentanți, Subversion și Git) găsiți aici [https://git.wiki.kernel.org/index.php/GitSvnComparison].

Git

Git [http://git-scm.com/] este unul dintre cele mai folosite sisteme de versionare distribuite. Cele mai importante concepte din Git sunt următoarele:

- repository - există un repository remote și oricâte repository-uri locale
- commit - conținut modificat
- branch - lanț de commit-uri

La începutul dezvoltării unui proiect folosind Git sunt necesare câteva configurări:

- numele, adresa de e-mail, editorul utilizat pentru mesajele de commit

```
$ git config --global user.name "Andrei Maruseac"
$ git config --global user.email "andrei.maruseac@gmail.com"
$ git config --global core.editor "vim"
```

Dacă nu este folosit argumentul –global, configurările sunt făcute doar pentru repository-ul curent.

- tipuri de fișiere care nu vor fi comise prin crearea unui fișier .gitignore. Un exemplu de astfel de fisier este:

```
$ cat .gitignore
*~
*.swp
*.swo
*.o
*.obj
*.a
*.so
*.dll
*.lib
*.gz
*.bz2
*.zip
```

Fișiere .gitignore specife limbajului de programare folosit puteți găsi pe GitHub [https://github.com/github/gitignore].

Comenzile Git folosite uzual în dezvoltarea proiectelor sunt:

- `man git-$comandă`, `man gittutorial` – pagini de manual pentru Git
- `git init` – inițializarea unui repository
- `git clone` – clonează un branch într-un director nou
- `git fetch` – preia commit-urile care nu există în repository-ul local
- `git merge` – integrează modificările din două branch-uri
- `git pull` – efectuează operațiile `fetch` și `merge` între branch-ul specificat și cel curent
- `git status` – arată ce fișiere au fost modificate de la ultimul commit și ce fișiere se află în staging area (se vor comite la comanda `git commit`)
- `git add` – adaugă fișiere în staging area
- `git commit` – comite modificările din staging area
- `git branch` – afișează toate branch-urile existente
- `git checkout` – dacă primește ca argument un branch, acesta devine branch-ul curent; dacă primește un nume de fișier ca parametru, se renunță la modificările de la ultimul commit din acest fișier
- `git push` – trimite modificările în repository-ul global
- `git log` – arată istoricul commit-urilor din branch-ul curent
- `git diff` – arată modificările față de un anumit commit sau între două commit-uri
- `git format-patch` – realizează patch-uri din ultimele commit-uri pentru a putea fi trimise prin e-mail
- `git am` – aplică patch-uri în branch-ul curent

Exerciții

Git what? (15 de minute)

Accesați platforma Kahoot [https://kahoot.it] și folosiți PIN-ul menționat de către asistent. Aceasta activitate are rolul de a vă testa rapid familiaritatea cu git.

Git (30 de minute)

Porniți în Linux și parcurgeți tutorialul Git Immersion [http://gitimmersion.com/]. Realizați primele 9 laboratoare din tutorial.

Dacă nu aveți instalat Git sau Ruby, din contul utilizatorului `root` instalați folosind `apt-get install git ruby rake`.

După realizarea tutorialului, creați un Repository al echipei. Repository-ul poate fi privat sau public, cum doriți, dar trebui să adăugați asistentul vostru la repository.

Lucru la proiect (60 de minute)

- **Obligatoriu** Creați un repository Git pe GitHub [https://github.com/]. Repository-ul poate fi privat sau public, cum doriți, dar trebui să adăugați asistentul vostru la repository.
- Stabiliti task-urile din WBS pentru proiectul vostru. Realizati de asemenea o diagrama Gantt utilizand unul dintre tool-urile mentionate (ex:Grafice Gantt [https://gantter.com/] sau Gantt Project [https://www.ganttproject.biz/])
- **Optional**Trello [https://trello.com/] pentru gestiunea task-urilor

mps/laboratoare/laborator-03.txt · Last modified: 2020/10/21 15:00 by iulia.stanica